



American International University-Bangladesh (AIUB)

Department of Computer Science

Faculty of Science & Technology (FST)

Project title: Integrated Sports Information and Management System for AIUB

Semester: Spring 25-26

Group: 04		Section: I
SL	Student Name	Student ID
1	Arghya Das	23-54149-3
2	Ananta Saha	23-54171-3
3	Ankon Saha	23-54172-3
4	Progga Patowary	22-47664-2

Table of Contents

1.	Project Proposal.....	XX
1.1	Background to the Problem	XX
1.2	Selection of Process Model	XX
2.	Timeline Chart / Gantt Chart	XX
3.	Software Requirements Specification (SRS) / PRD	XX
3.1	Scopes and Features.....	XX
3.2	User Story Table	XX
3.3	Requirements Traceability Matrix	XX
3.3.1	Functional Requirements	XX
3.3.2	Non-Functional Requirements.....	XX
4.	Software Design	XX
4.1	System Design	XX
4.2	UI Design using Figma.....	XX
5.	Git Workflow	XX
6.	Software Testing	XX
7.	Conclusion	XX

1. PROJECT PROPOSAL

1.1 Background to the Problem

American International University-Bangladesh organizes many sports activities and tournaments every year. These sports events help students stay active and healthy. Sports also help students learn teamwork, discipline, and leadership.

However, the current system for sports information is not very good. Most sports information is shared through Facebook posts or official notices. Because of this, many students miss important information. Students often do not know the match schedule, tournament registration date, or gym membership notice.

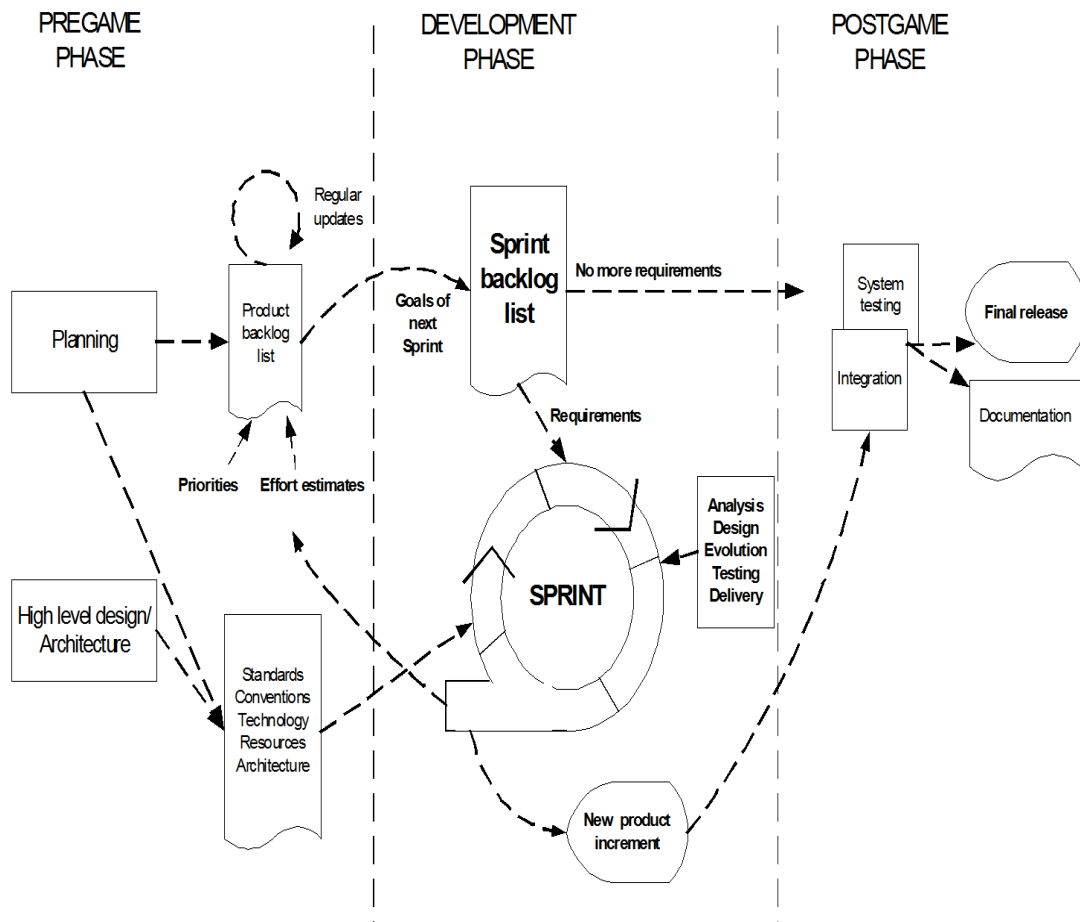
Another problem is that students must go to the sports office to book sports slots or get information. This process takes time and is not easy for students. Sometimes there is also confusion about schedules and event updates.

Because there is no central digital system, the information is not well organized. Students face problems getting information quickly. The sports department also finds it difficult to manage records and event data.

For this reason, a digital system is needed. The Integrated Sports Information and Management System will help manage sports information in one place. Students and administrators will be able to use the system easily.

1.2 Selection of Process Model

For this project, the **Agile Scrum model** will be used.



Agile Scrum is a modern software development method. In this model, the system is developed step by step. Each step is called a **Sprint**. A sprint usually takes one to four weeks.

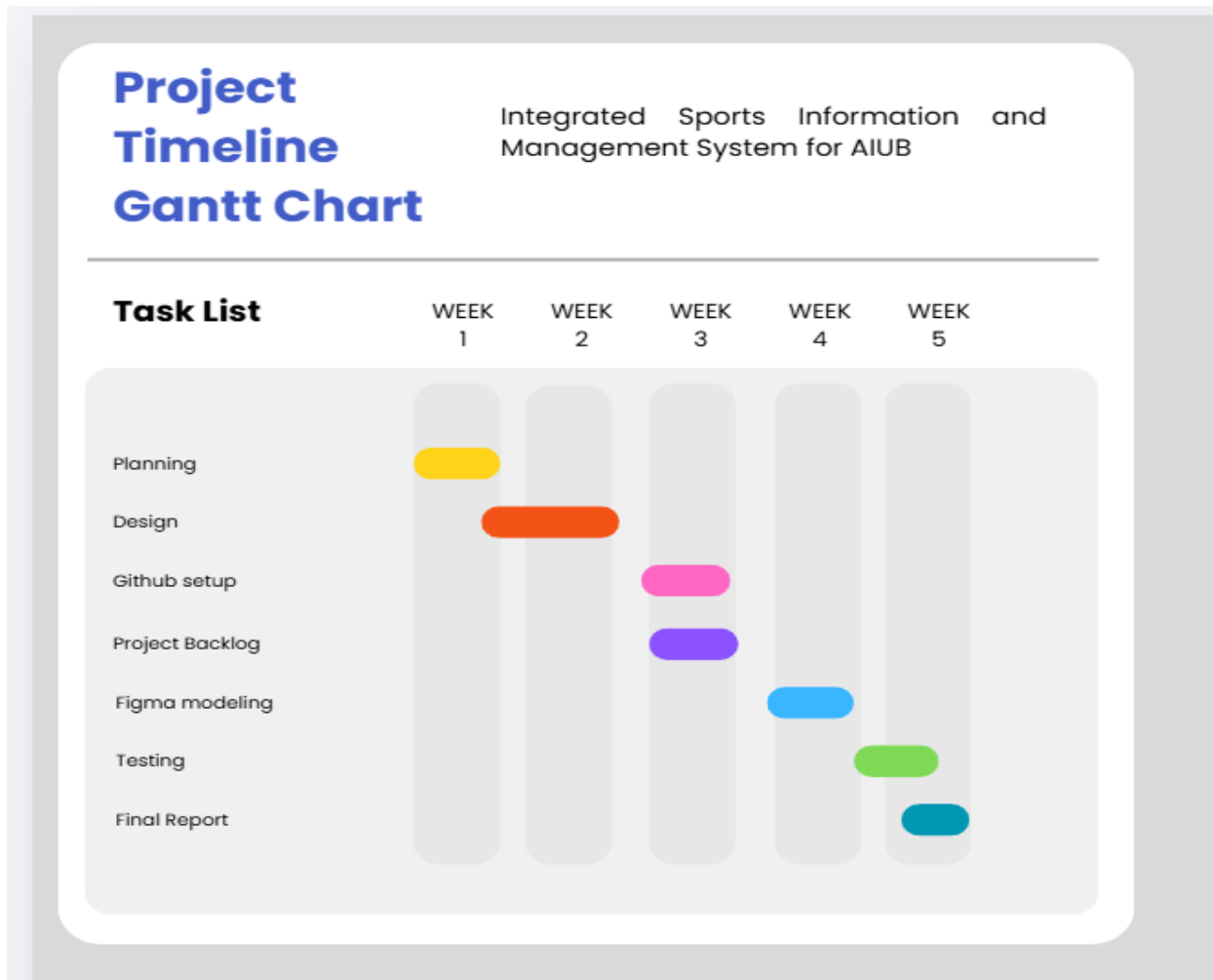
In every sprint, a small part of the system is developed and tested. After that, feedback is taken and improvements are made.

For this project, the development may include several sprints such as:

- User registration system
- Sports event management
- Tournament registration system
- Sports slot booking system
- Announcement and notification system

Using Scrum helps the team work in an organized way and improve the system step by step.

2. Timeline Chart / Gantt Chart



3. SOFTWARE REQUIREMENTS SPECIFICATIONS (SRS) / PRODUCT REQUIREMENTS DOCUMENT (PRD)

3.1 Scopes and Features

Scope 1: User Account Management

Covers all functions for user registration, login, and account settings.

- User Registration (Student and Admin) with email, name, gender, date of birth, password, and role.
- User Login (Sign In) with email and password, with a Remember Me option.
- Password Recovery via email OTP or recovery link.
- View Profile: display name, student ID, email, phone, and department.
- Edit Profile: update name, email, phone, date of birth, and gender.
- Change Profile Photo: upload a new profile image (PNG or JPG only).
- Change Password: enter current password and set a new secure password.

Scope 2: Home Dashboard

Provides a central landing page for all users after login.

- Student Home: request membership, book slots, view schedule, see current and upcoming events.
- Admin Home: approve membership, create slots, accept slot requests, create schedule, add events.
- General Dashboard: display available slots, upcoming events, current events, and available seats.

Scope 3: Event Management

Allows the admin to manage sports events and users to view them.

- Manage Events (Admin): create, edit, and delete sports events with name, type, date, time, venue, and description.
- Upcoming Events (User): view list of future events sorted by closest date.
- Current Events (User): view only today's events with their status.

Scope 4: Payment Management

Handles online payment submission by users and confirmation by admins.

- Make Payment (User): enter recipient address, amount, and sign with a private key.
- Confirm Payment (Admin): review pending payments and accept or reject them.

Scope 5: Performance Management

Tracks and visualizes player and team performance statistics.

- Performance Tracking Input (Admin): enter player ID, sport, match date, and dynamic stats.
- Performance Dashboard (Admin/Coach): filter by sport and team, view charts, export reports.

Scope 6: Health and Wellness Module

Manages player health records, injuries, and medical information.

- Injury Logging and Tracking (Admin): log injury type, date, recovery status, and estimated return date.
- Student Medical Profile (Student/Admin): store emergency contact, allergies, and current medications.

Scope 7: Equipment Management

Maintains a digital inventory of sports equipment and loan records.

- Equipment Catalogue and Stock (Admin): add, edit, and delete equipment with name, category, quantity, and condition.
- Equipment Loan and Return (Admin): log equipment loans, deduct stock, and update stock on return.

Scope 8: Discipline Management

Records student misconduct and manages disciplinary actions.

- Incident Report (Admin): record student ID, incident type, date, reporter, and description.
- Disciplinary Action and Sanction (Admin): assign suspension or ban linked to an incident report.

Scope 9: Inter College Tournaments

Manages inter-college sports events including registration and results.

- Inter College Home (Admin/User): display current champions and runners-up per sport.
- Registration for Upcoming Game (User): fill and submit team registration form before the deadline.
- Manage Registration (Admin): open or close registration, customize form fields, and connect to a data sheet.
- Inter College Cricket, Football, Basketball Pages (Admin/User): create photo albums and view past tournament photos.

Scope 10: Achievements

Records and displays sports achievements for students and admins.

- All Achievements (Admin/User): admin adds, edits, or deletes achievements; users view achievements by sport.

Scope 11: Coach and Trainer Management

Manages coaching staff profiles and assignments.

- Coach Profile Management (Admin): create coach profile with name, contact, sport category, and experience.
- Coach Assignment to Teams and Events (Admin): assign coaches to teams and events.

Scope 12: Team Formation and Squad Management

Organizes team creation, player assignment, and squad finalization.

- Team Creation and Player Assignment (Admin): create a team for a tournament and assign registered players.
- Role Assignment and Squad Finalization (Admin/Coach): assign captain, positions, and jersey numbers.

Scope 13: Facilities and Venue Management

Manages sports facility information and venue booking.

- Facility and Venue Information Management (Admin): add facility name, type, capacity, opening time, and status.
- Venue Availability and Assignment Management (Admin): assign venues to events, check conflicts, and prevent double booking.

Scope 14: Budget and Expense Tracking

Tracks financial planning and spending for sports events.

- Budget Allocation (Admin): assign a budget amount to a specific event or tournament.
- Expense Recording and Monitoring (Admin): record expense category, amount, date, and optional receipt.

Scope 15: Sports Calendar and Smart Scheduling

Provides a visual calendar and conflict management for sports activities.

- Event and Training Schedule Creation (Admin): create schedules with sport type, event type, date, time, and venue.
- Schedule Viewing and Conflict Management (Admin/User): view monthly or weekly calendar, filter by sport, and get conflict alerts.

Scope 16: Renowned Players

Displays profiles of well-known players associated with AIUB sports.

- Renowned Players Page (User): view player photo, name, country, sport category, and full profile.

3.2 User Story Table

As a/an	I want to	So that	Acceptance Criteria
Student / Admin	sign up with my email, name, and password	I can create an account and log in later	Email must be unique and valid. Password must be at least 8 characters. Role must be selected. User is redirected to dashboard after signup.
Student / Admin	sign in with my email and password	I can access my dashboard	Email and password must match the database. Wrong credentials show an error message. User is redirected to the correct dashboard.
Student / Admin	recover my password using my email	I can reset my password if I forget it	Email must exist in the database. A recovery code is sent to the email. New password must be at least 8 characters with one special character.
Student / Admin	view my profile information	I can see my name, ID, email, phone, and department	No null values are shown. An edit button is visible and takes the user to the edit page.
Student / Admin	edit my profile details	I can keep my personal information up to date	Email must be in valid format. Phone must be 11 digits. Date of birth must be

As a/an	I want to	So that	Acceptance Criteria
			at least 15 years old. A success or error message is shown on save.
Student / Admin	change my profile photo	my profile looks up to date	Only PNG or JPG files are accepted. After confirming, the user is redirected to the view profile page.
Student / Admin	change my password	my account stays secure	Current password must match the database. New password must be at least 8 characters. Confirm password must match. Success or error message is shown.
Student	see available slots, events, and membership options on the home page	I can quickly access sports services	User must be logged in. Available slots and events are displayed correctly. Membership request option is visible.
Admin	see membership requests, slot management, and event options on the home page	I can manage sports operations quickly	Admin must be logged in. Pending membership requests are visible. Admin can accept or reject requests.
Admin	create, edit, and delete sports events	students stay informed about upcoming activities	Admin must be logged in. Event name, date, and place are required. Events appear in the list after creation.
Student	view upcoming sports events	I can plan my participation in advance	Only future events are shown. Events are sorted by closest date. No login is required to view event details.
Student	see current events for today	I know which sports activities are happening now	Only today's events are shown. Events automatically move to past status after the date ends. Status shows as Ongoing.
Student	make an online payment for sports fees	I can pay without visiting the office	Recipient address must be valid. Amount must be a positive number not more than the user's balance. User must sign with a private key before submitting.

As a/an	I want to	So that	Acceptance Criteria
Admin	confirm or reject student payments	financial records stay accurate	Payment must have a reference number. Status is Pending until admin acts. Confirmed payments instantly update the student's payment status.
Admin	enter player performance statistics for a match	I can track player performance over time	Player ID must exist in the database. Sport and match date are required before saving.
Admin / Coach	view and filter performance data on a dashboard	I can identify trends and make coaching decisions	Dashboard updates in real time based on filters. Charts and player data are shown correctly. Export report button works.
Admin / Coach	log a player injury with type and recovery dates	I can manage player availability and safety	Player ID must exist in the database. Date logic must be correct. Recovery status is updated after saving.
Student	save my emergency contact and medical information	coaches and admins can access it in an emergency	Contact number must be in valid numeric format. Medical profile is only accessible by the specific student and admin.
Admin	maintain a digital list of all sports equipment	I can track stock and manage inventory easily	Clicking Add New Item opens a form with name and quantity. Loan Out deducts from available stock. Stock is updated immediately.
Admin	log equipment loans and returns	I know what equipment is in use and what is available	Loan Out deducts the given quantity. Record Return adds it back and timestamps the return. System prevents loan if quantity exceeds available stock.
Admin	record student misconduct incidents digitally	there is consistent and accurate documentation	Player ID must exist. Incident Type and Reporting Staff Name are required. A unique Incident ID is generated on save.
Admin	assign and track disciplinary actions from an incident	sanctions are enforced and communicated clearly	Admin must select a valid Incident Report ID. Dates must be logically correct. Clicking Confirm Action logs the sanction and notifies stakeholders.

As a/an	I want to	So that	Acceptance Criteria
Admin	add or update current champions and runners-up per sport	the system reflects the latest inter-college results	Sport and tournament must be selected before saving. Champion and Runner-up fields cannot be empty. Same team cannot be both Champion and Runner-up.
Student	register my team for an upcoming inter-college tournament	my team can participate in the event	Registration is only allowed before the deadline. Team name, college name, captain name, email, and phone are required. Email and phone must be in valid format.
Admin	open or close registration and customize form fields	the registration process is well-controlled	Admin can set start and end dates for registration. Registration closes automatically after the end date. Form changes are visible to users in real time.
Admin	add, edit, and delete sports achievements	the achievement records stay accurate and current	Achievement title, tournament name, year, and position are required. System validates the year format. Uploaded images must be JPG or PNG.
Student	view all sports achievements by category	I can see what AIUB teams and players have accomplished	Achievements are grouped by sport. All entries are displayed without requiring login.
Admin	create and manage coach profiles	coaching staff information is properly maintained	Coach name and sport category are required. Contact must be in valid format. Each coach profile has a unique Coach ID.
Admin	assign coaches to teams and events	training and tournaments are supervised effectively	Selected coach must exist in the system. Duplicate coach assignments are prevented. Assigned teams and events appear in the coach profile.
Admin	create teams and assign players for tournaments	participation is properly organized	Tournament must exist. Team name must be unique within the tournament. A player cannot be on more than one team in the same tournament. A unique Team ID is generated.

As a/an	I want to	So that	Acceptance Criteria
Admin / Coach	assign roles and finalize team squads	each team is ready for official participation	Each team must have one captain. Player Position and Jersey Number are required. Jersey numbers must be unique within the team. Finalized squads cannot be changed without admin permission.
Admin	add and manage detailed sports facility information	all infrastructure details are stored digitally	Facility name and type are required. Facility name must be unique. Opening time must be earlier than closing time. A unique Facility ID is generated on save.
Admin	assign venues to sports events	scheduling conflicts are avoided	Facility must be marked as available. Usage date, start time, and end time are required. System prevents double booking. Conflicts trigger a warning message.
Admin	allocate a budget to a sports event	expenses can be planned properly	Event must exist. Budget amount must be a positive number. Only one budget can be allocated per event. A unique Budget ID is generated.
Admin	record and monitor expenses for an event	financial tracking stays transparent	Event and expense category must be selected. Amount must be positive. Expense date is required. An optional receipt file can be uploaded.
Admin	create schedules for sports events and training sessions	all activities are planned in advance	Sport type, event type, date, start time, end time, and venue are required. Start time must be before end time. Venue must be available. A unique Schedule ID is generated.
Admin / Student	view schedules in a calendar and get conflict alerts	I can avoid overlapping bookings	All events are shown in monthly and weekly view. Users can filter by sport type. Overlapping schedules for the same venue trigger a warning. Updated schedules are reflected immediately.
Student	view the photo and profile of renowned players	I can learn about notable AIUB sports	A high-quality photo must load. If the image is broken, show: Image not available. Player name, country, and

As a/an	I want to	So that	Acceptance Criteria
		alumni and national players	sport category must be visible. View Profile opens the full player profile page.
System	perform daily automated backups of registration data	data is not lost in case of failure	Backup runs automatically every day. Backup covers all registration data. Backup completion is logged.
System	detect and alert about scheduling conflicts for the same venue	admins can fix overlapping bookings before they cause problems	System compares venue usage dates and times. A conflict message is shown immediately when a conflict is detected.

3.3 Requirements Traceability Matrix

3.3.1 Functional Requirements

Req. ID	Module Name	Requirement Description
FR-1	User Registration	The system shall allow students and admins to register using a unique email, name, password (min 8 characters), gender, date of birth, and role selection.
FR-2	User Login	The system shall allow registered users to log in using a valid email and password. Wrong credentials shall display an error message.
FR-3	Password Recovery	The system shall send a recovery code to the user's email. The user shall be able to set a new password of at least 8 characters with one special character.
FR-4	View Profile	The system shall display the user's name, student ID, email, phone number, and department. No null values shall be shown.

Req. ID	Module Name	Requirement Description
FR-5	Edit Profile	The system shall allow users to update their name, email, phone (11 digits), and date of birth. Email must follow a valid format (xyz@abc.com).
FR-6	Change Profile Photo	The system shall allow users to upload a new profile photo in PNG or JPG format only.
FR-7	Change Password	The system shall verify the current password before allowing the user to set a new password. The new password must be confirmed by re-entry.
FR-8	Student Home Dashboard	The system shall display available slots, membership request option, current events, and upcoming events to logged-in students.
FR-9	Admin Home Dashboard	The system shall allow admins to approve memberships, create slots, accept slot requests, and create events from the home page.
FR-10	Manage Events	The system shall allow admins to create, edit, and delete sports events. Event name, date, and venue are required fields.
FR-11	Upcoming Events	The system shall display only future events sorted by closest date. No login shall be required to view event details.
FR-12	Current Events	The system shall display only today's events. Events shall automatically move to past status after the event date ends.
FR-13	Make Payment	The system shall allow students to make payments by entering a valid recipient address, amount (positive, not more than balance), and signing with a private key.
FR-14	Confirm Payment	The system shall allow admins to confirm or reject payments. Payment status shall remain Pending until the admin acts.
FR-15	Performance Tracking Input	The system shall allow admins to enter player statistics by selecting player ID, sport, and match date. Player ID must exist in the database.
FR-16	Performance Dashboard	The system shall display performance data in charts that update in real time based on sport and team filter selections. Admins shall be able to export reports.

Req. ID	Module Name	Requirement Description
FR-17	Injury Logging and Tracking	The system shall allow admins to log player injuries with type, date, recovery status, and estimated return date. Player ID must exist in the database.
FR-18	Student Medical Profile	The system shall allow students to save emergency contact, allergies, and medications. Access shall be restricted to the specific student and admins only.
FR-19	Equipment Catalogue and Stock	The system shall allow admins to add, edit, and delete equipment items with name, category, quantity, and condition. Available stock shall update immediately.
FR-20	Equipment Loan and Return	The system shall deduct stock when equipment is loaned out. It shall add stock back when equipment is returned. The system shall prevent loans that exceed available stock.
FR-21	Incident Report	The system shall allow admins to record student misconduct with incident type, date, reporter name, and description. A unique Incident ID shall be generated on save.
FR-22	Disciplinary Action and Sanction	The system shall allow admins to assign sanctions linked to an Incident Report ID. Dates must be logically valid. The system shall notify relevant stakeholders on confirmation.
FR-23	Inter College Home	The system shall allow admins to update current champions and runners-up per sport. Users shall be able to view the latest champion information.
FR-24	Inter College Registration	The system shall allow students to register a team before the registration deadline. Team name, college name, captain name, email, and phone are required fields.
FR-25	Manage Registration	The system shall allow admins to open and close registration, set start and end dates, and customize form fields. Registration shall close automatically after the end date.
FR-26	Tournament Photo Albums	The system shall allow admins to create photo albums for inter-college tournaments. Students shall be able to view past tournament photos without editing permissions.

Req. ID	Module Name	Requirement Description
FR-27	Achievements Management	The system shall allow admins to add, edit, and delete achievements. Achievement title, tournament name, year, and position are required. Images must be JPG or PNG.
FR-28	Coach Profile Management	The system shall allow admins to create coach profiles with name, contact information, sport category, and experience. Each coach shall have a unique Coach ID.
FR-29	Coach Assignment	The system shall allow admins to assign coaches to teams and events. Duplicate assignments shall be prevented. Assigned items shall appear in the coach profile.
FR-30	Team Creation and Player Assignment	The system shall allow admins to create teams for a tournament and assign registered players. Team name must be unique within the same tournament. A unique Team ID shall be generated.
FR-31	Role Assignment and Squad Finalization	The system shall allow admins or coaches to assign a captain, player positions, and jersey numbers. Jersey numbers must be unique within the team. Finalized squads shall require admin permission to modify.
FR-32	Facility and Venue Information Management	The system shall allow admins to add facilities with name, type, capacity, supported sports, opening and closing times, and status. A unique Facility ID shall be generated on save.
FR-33	Venue Availability and Assignment	The system shall prevent double booking of the same venue. Conflicts shall trigger a warning message. A unique Assignment ID shall be generated after venue assignment.
FR-34	Budget Allocation	The system shall allow admins to allocate a budget to a specific event. Budget amount must be a positive number. Only one budget can be allocated per event. A unique Budget ID shall be generated.
FR-35	Expense Recording and Monitoring	The system shall allow admins to record expenses by event with category, amount, date, and optional receipt upload. Expense records shall be linked to an allocated budget.
FR-36	Event and Training Schedule Creation	The system shall allow admins to create schedules with sport type, event type, date, start time, end time, and venue. Start time must be before end time. Venue must be available. A unique Schedule ID shall be generated.

Req. ID	Module Name	Requirement Description
FR-37	Schedule Viewing and Conflict Management	The system shall display all scheduled events in a monthly and weekly calendar view. It shall detect overlapping schedules for the same venue and show a warning message.
FR-38	Renowned Players	The system shall display renowned players with their photo, name, country, and sport category. If the image fails to load, the system shall show: Image not available. A View Profile button shall open the full player profile.
FR-39	Automated Data Backup	The system shall perform daily automated backups of all registration data. Backup completion shall be logged.

3.3.2 Non-Functional Requirements

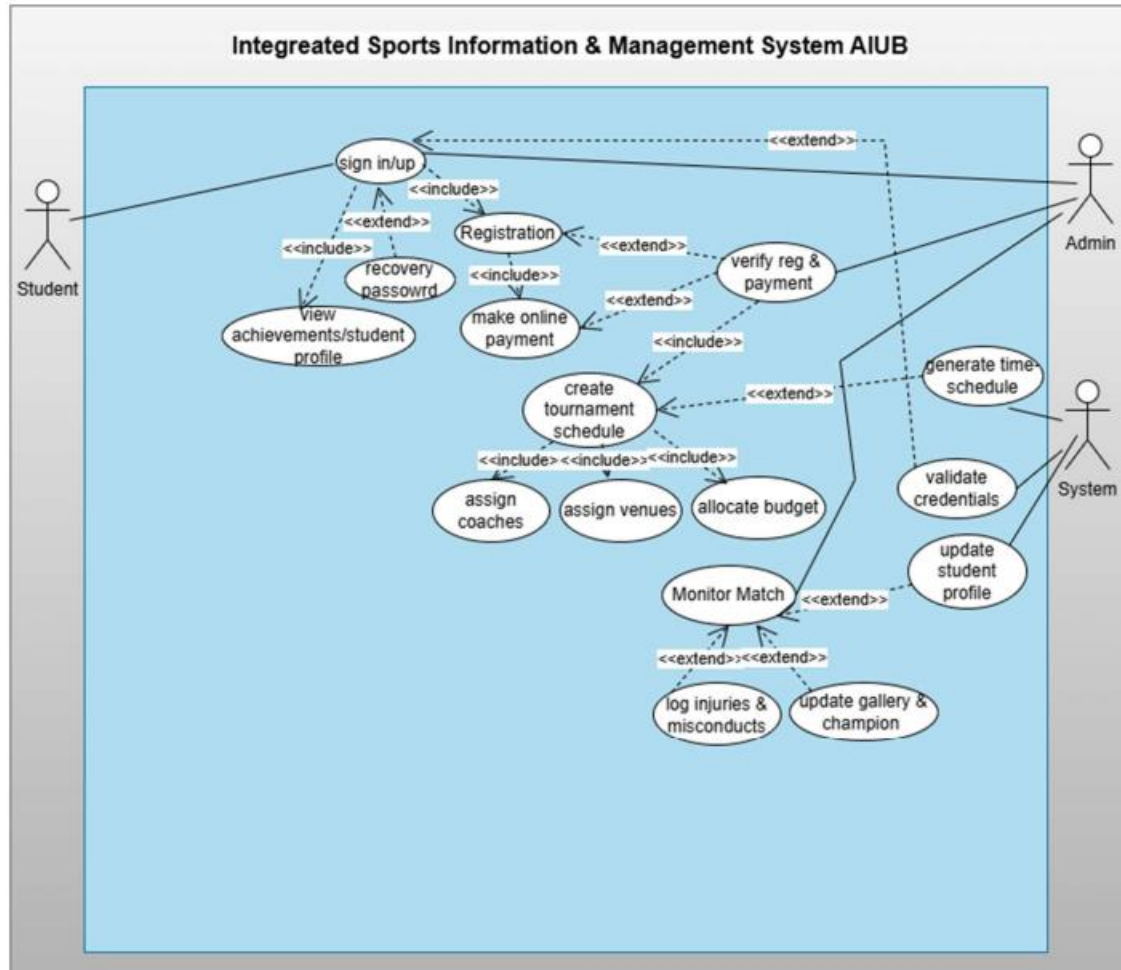
Req. ID	Category	Requirement Description
NFR-1	Performance — Page Load	The system shall load the home page within 3 seconds under normal network conditions.
NFR-2	Performance — Form Submission	The system shall complete registration form submission within 5 seconds after clicking Register.
NFR-3	Performance — Concurrent Users	The system shall support at least 200 concurrent users during peak hours without slowdown.
NFR-4	Performance — Data Volume	The system shall handle at least 5,000 total registrations without crashing or slowing significantly.
NFR-5	Security — HTTPS	The system shall use HTTPS (SSL/TLS) to secure all data transmissions.
NFR-6	Security — Password Hashing	User and admin passwords shall be stored using secure hashing (bcrypt or SHA-256). Passwords shall never be stored as plain text.
NFR-7	Security — Injection Prevention	The system shall prevent SQL Injection and Cross-Site Scripting (XSS) attacks on all input fields.

Req. ID	Category	Requirement Description
NFR-8	Security — Access Control	The system shall restrict access so that only authorized admins can view or edit registrations, student medical profiles, and financial records.
NFR-9	Usability — Ease of Use	The user interface shall be simple enough that a first-time user can complete tournament registration within 3 minutes.
NFR-10	Usability — Error Messages	The system shall show clear and specific error messages for all invalid or missing input fields.
NFR-11	Usability — UI Consistency	All buttons, fonts, and page layouts shall be consistent across all pages of the system.
NFR-12	Usability — Language	The system shall support English language content with readable fonts and proper text alignment.
NFR-13	Reliability — Uptime	The system shall provide at least 99% uptime during the tournament registration period.
NFR-14	Reliability — Data Protection	The system shall not lose already-entered registration data when a user refreshes the page accidentally. Auto-save or an unsaved changes warning shall be implemented.
NFR-15	Reliability — Maintenance Message	The system shall display a proper maintenance message if the site goes down temporarily.
NFR-16	Scalability — New Sports	The system shall be scalable to support additional sports (for example, Volleyball and Badminton) without requiring a major redesign.
NFR-17	Scalability — Data Growth	The database shall support increased data volume for players, teams, and tournaments in future years without performance loss.
NFR-18	Compatibility — Browsers	The system shall work correctly on modern browsers: Chrome, Firefox, Edge, and Safari.
NFR-19	Compatibility — Responsiveness	The system shall be fully usable on mobile, tablet, and desktop screens with a responsive layout.
NFR-20	Maintainability — Backup	The system shall perform daily automated backups of all registration data to prevent data loss.

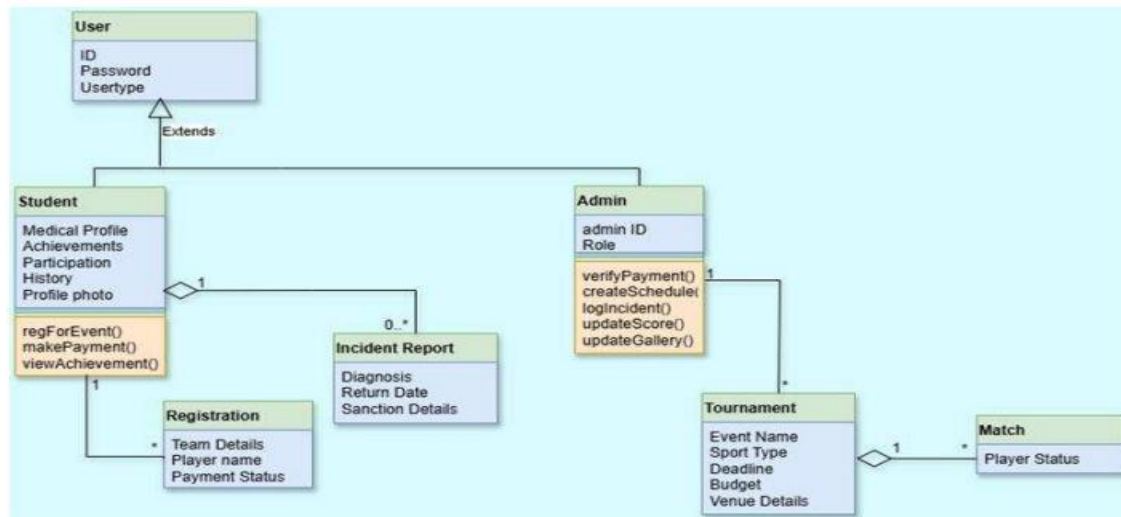
4. SOFTWARE DESIGN

4.1 System Design: Draw the system design for your project using **Draw.io** / **MS tools**.

Use Case Diagram:

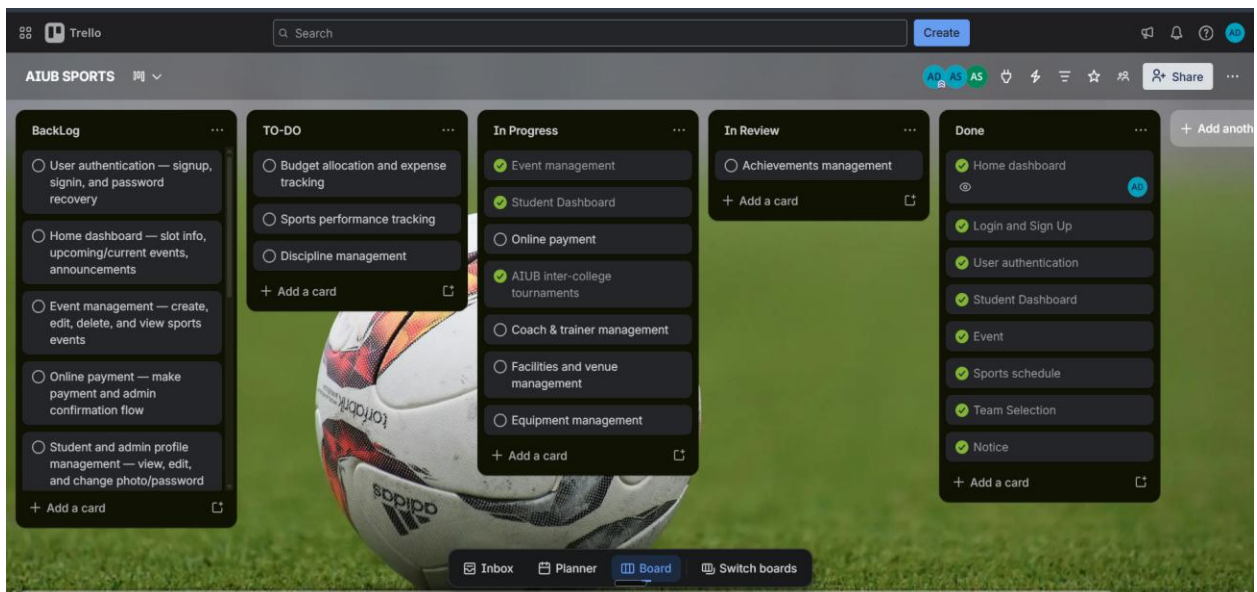


Class Diagram:

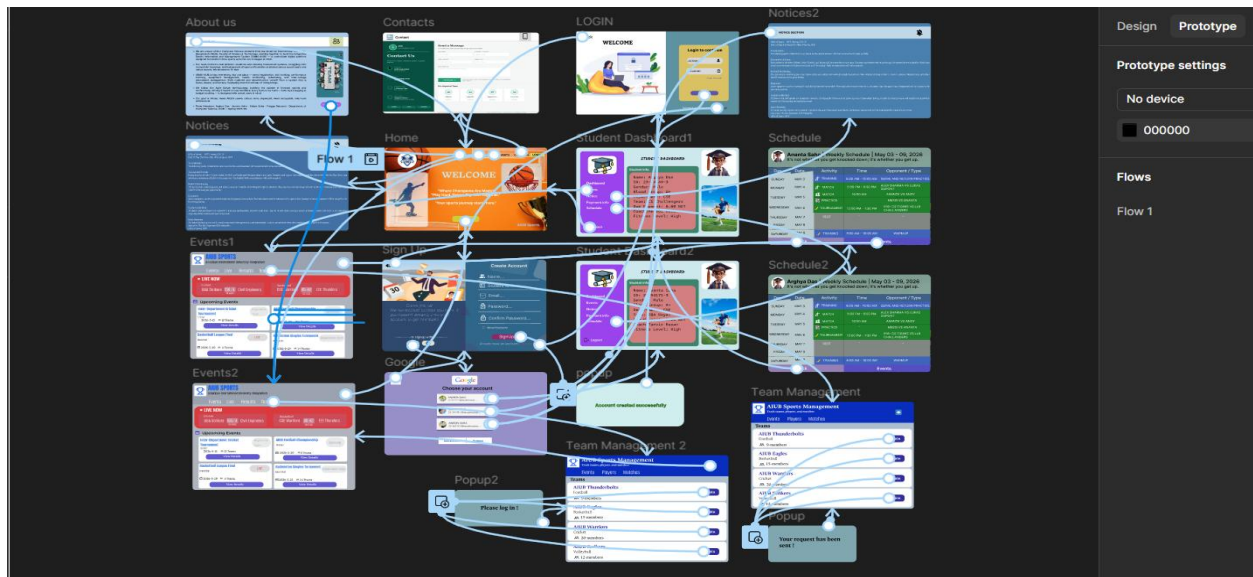


4.2 UI / Wireframe Design using Figma

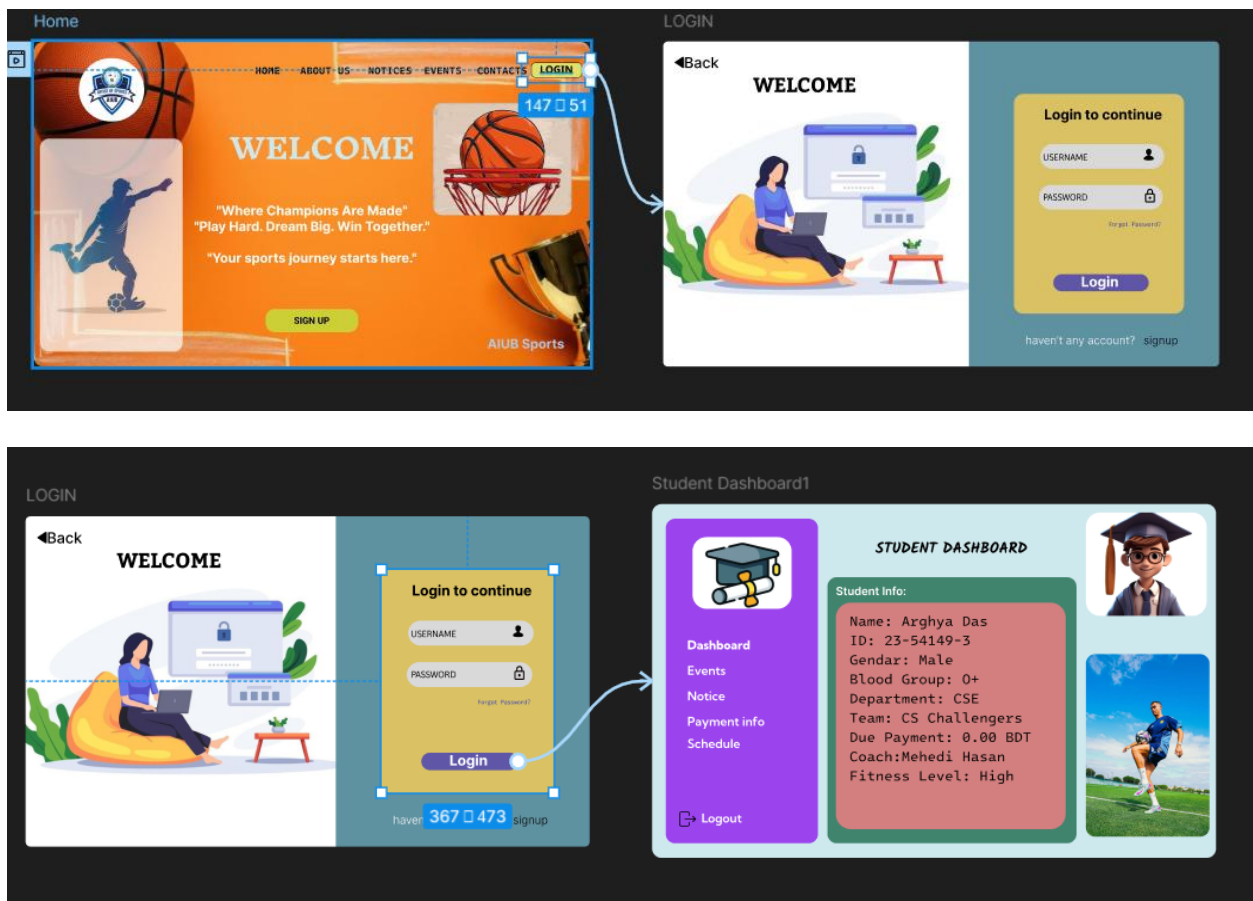
Trello:

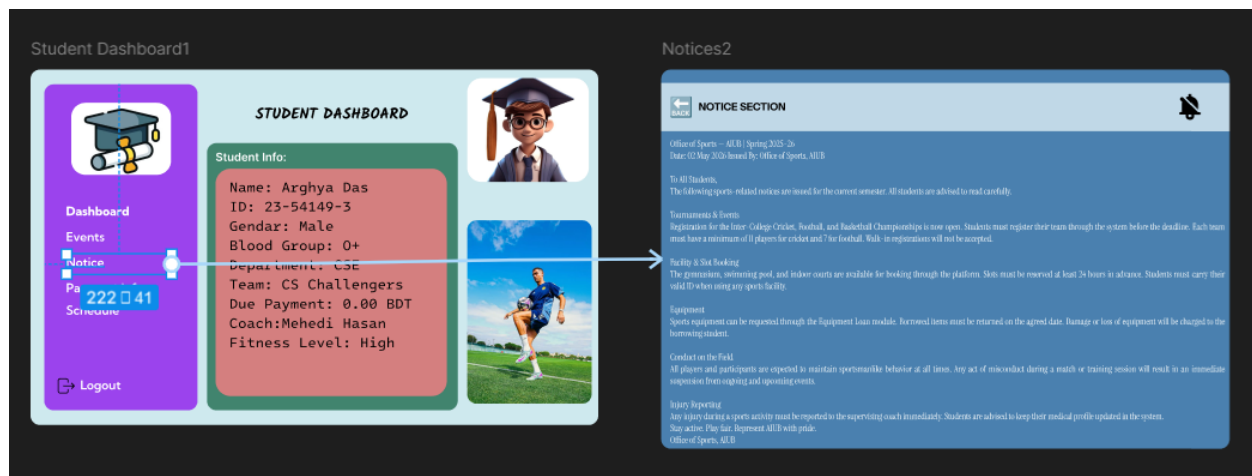


UI Design by Figma:

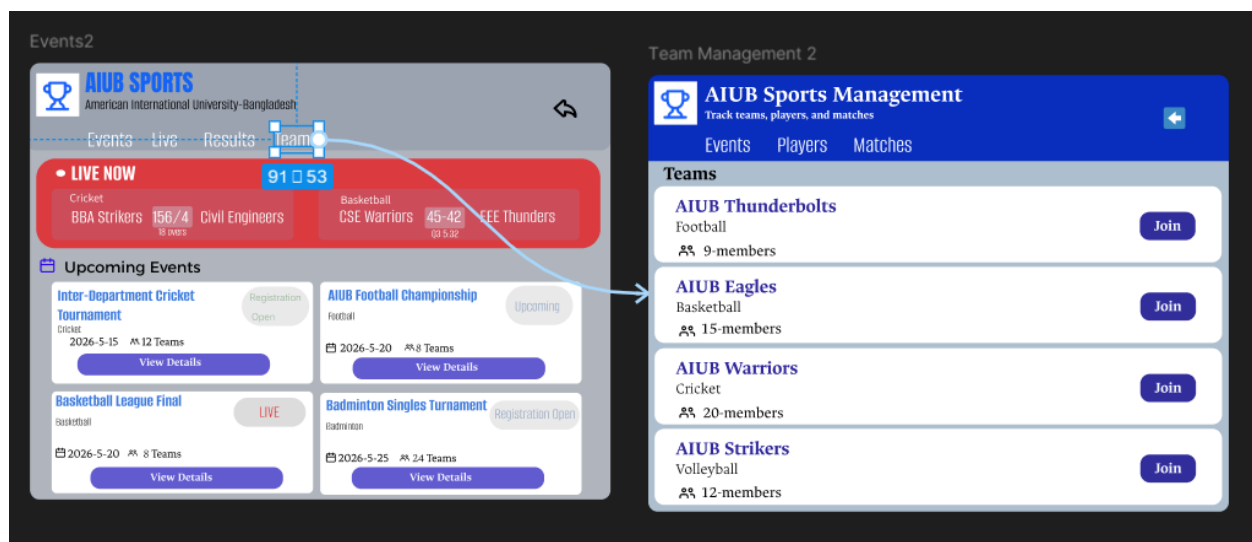
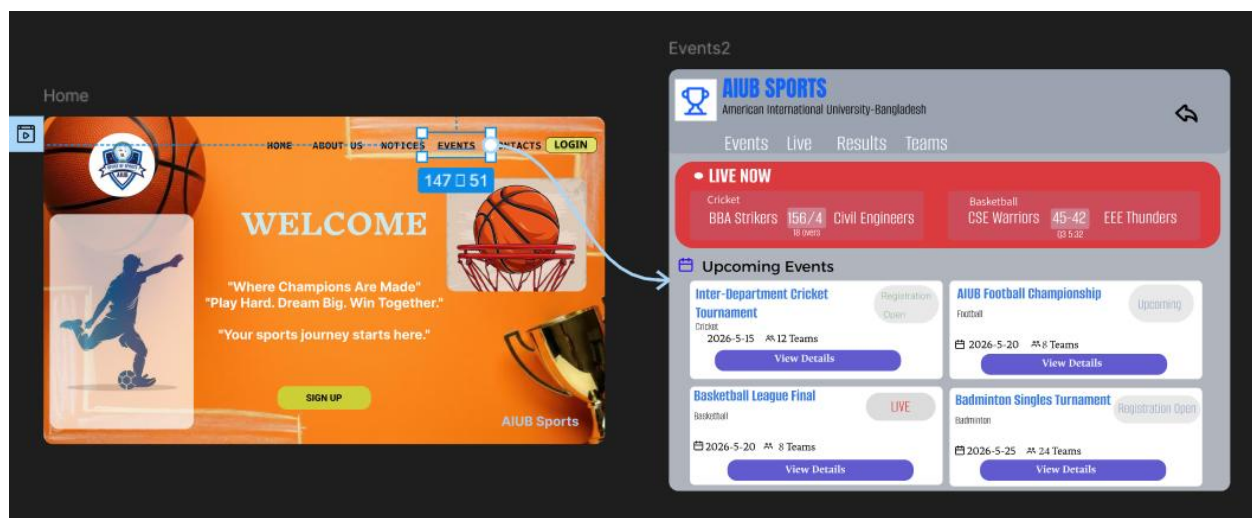


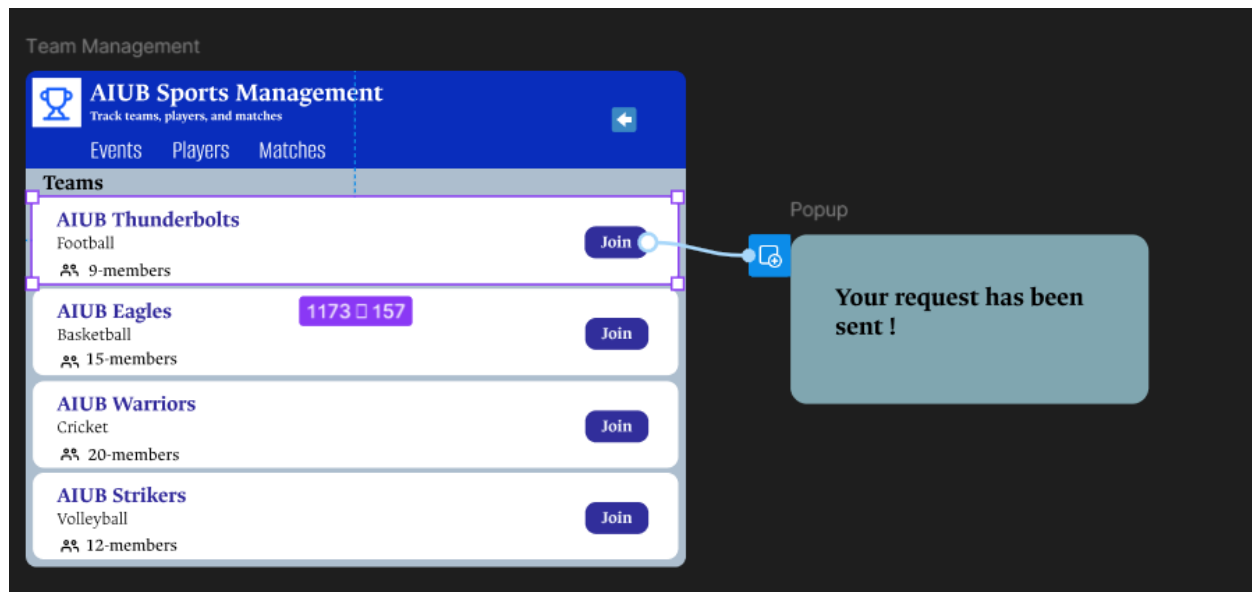
Student-1 : Arghya Das (23-54149-3)



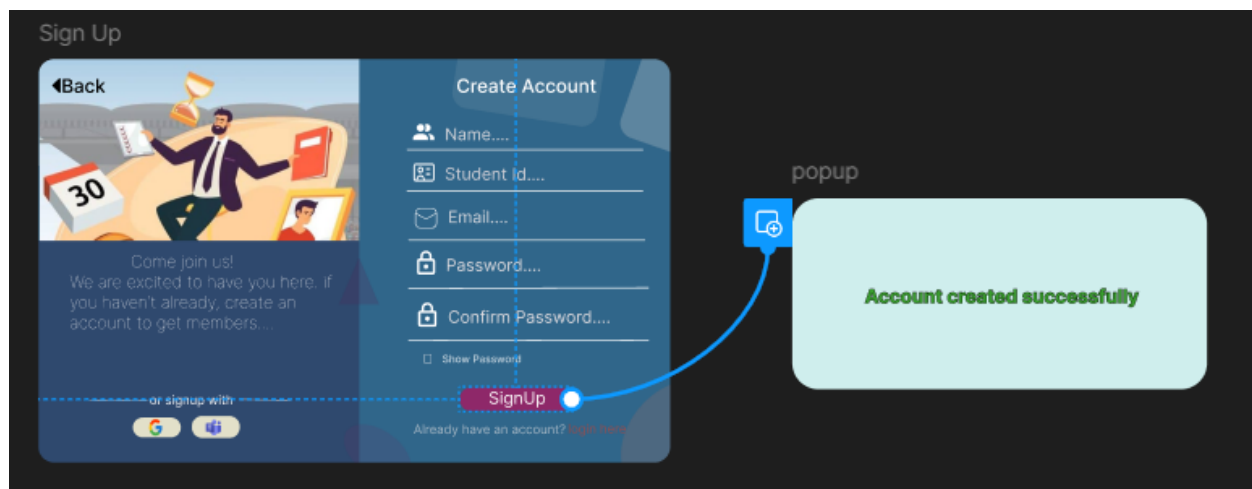
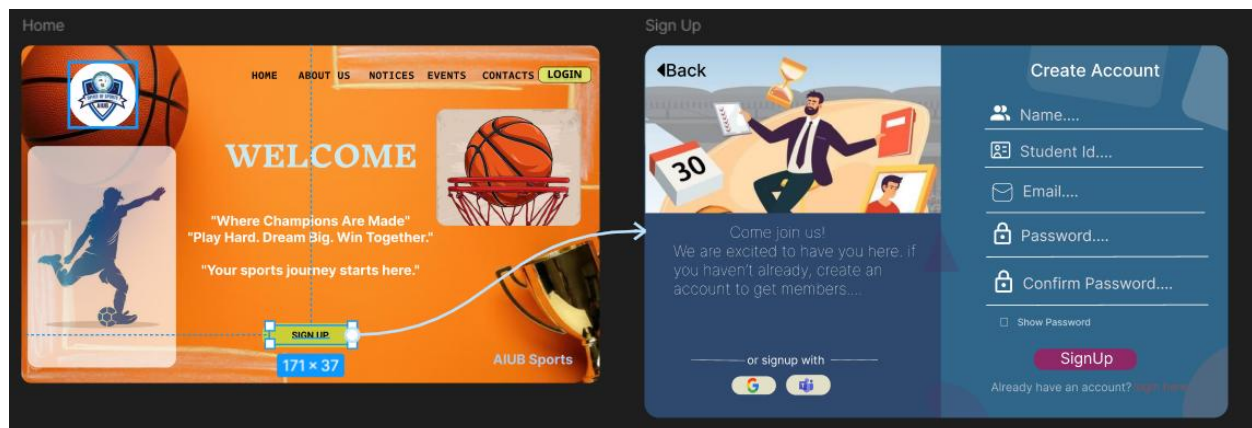


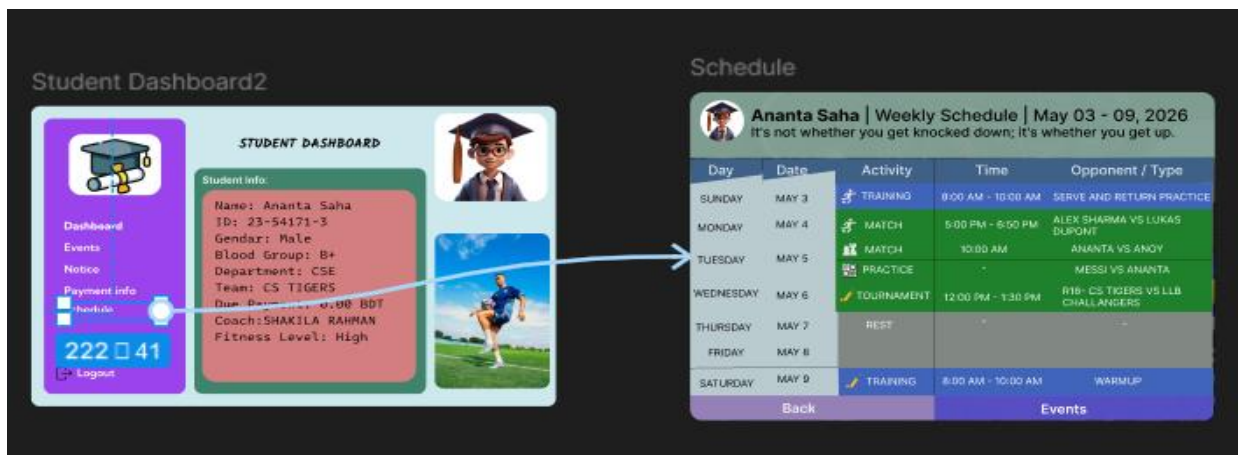
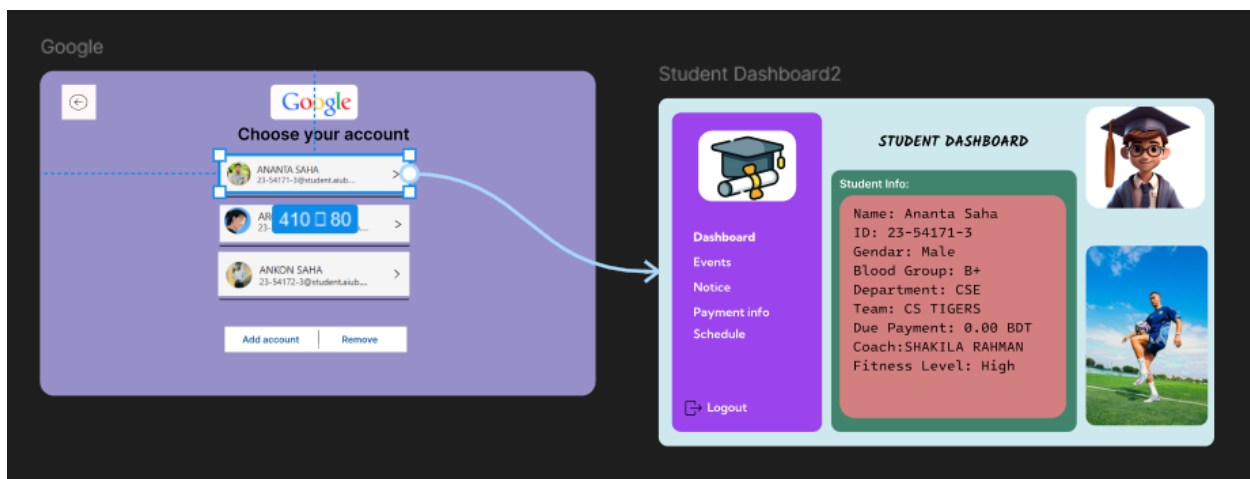
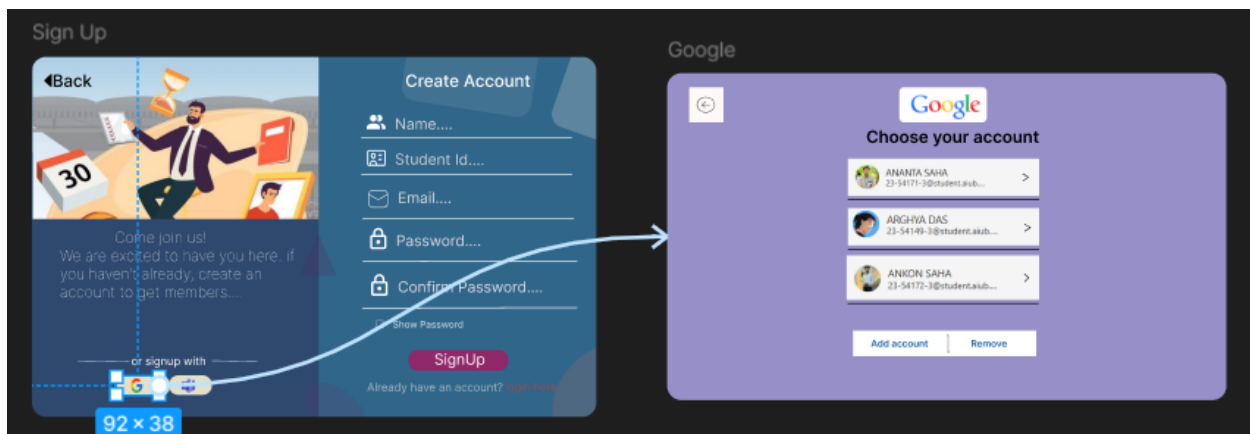
Student-2 : Ankon Saha (23-54172-3)



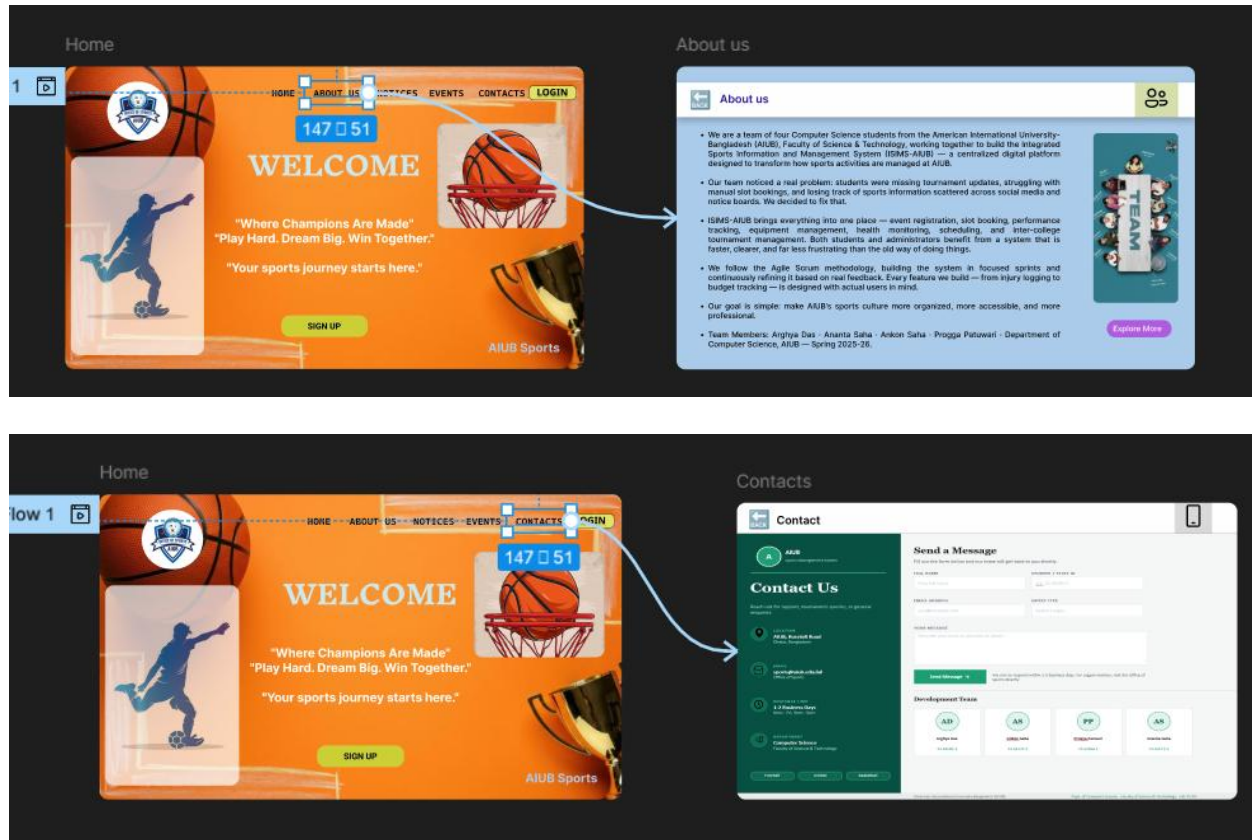


Student-3 : Ananta Saha(23-54171-3)





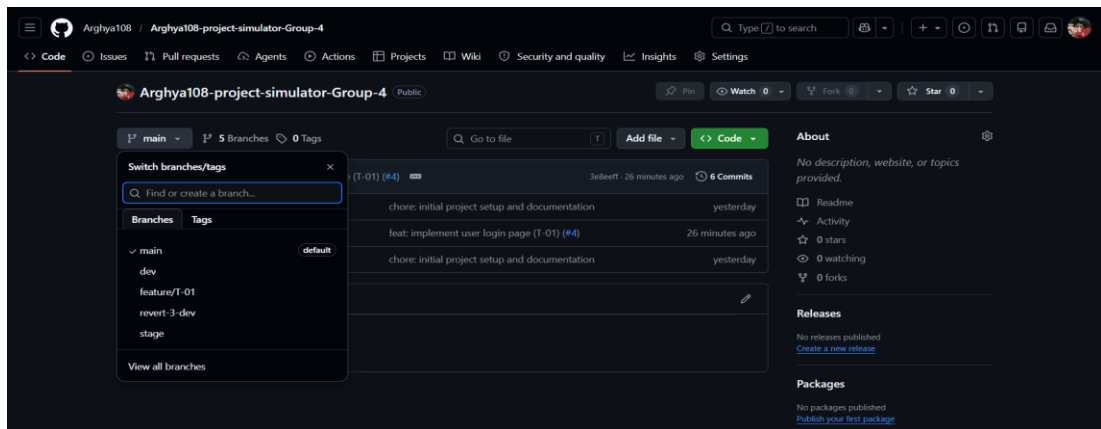
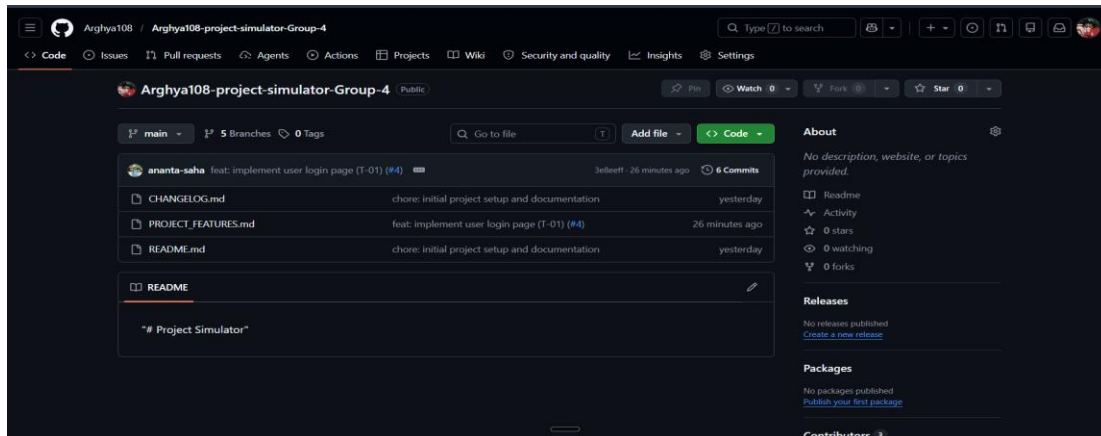
Student-4 : Progga Patowary (22-47664-2)



Figma Link: <https://www.figma.com/design/dBikoEuOFBS0pXbbmmVzQG/Integrated-Sports-Information-And-Management--System.?node-id=0-1&t=e2CY5gUQBoF4q0CX-1>

5. GIT WORKFLOW

Student-1: Repo Create [Arghya Das (23-54149-3)]



Student-2: Clone Repo [Ananta Saha (23-54171-3)]

```
MINGW64/C:/Users/hp/project-simulator/Arghya108-project-simulator-Group-4
$ git branch -d feature/T-14
error: cannot delete branch 'feature/T-14' used by worktree at 'C:/Users/hp/project-simulator/Arghya108-project-simulator-Group-4'

$ git checkout dev
Switched to branch 'dev'
Your branch is up to date with 'origin/dev'.

$ git branch -d feature/T-14
Deleted branch feature/T-14 (was 453fa81).

$ git checkout -b feature/T-01
Switched to a new branch 'feature/T-01'

$ git add PROJECT_FEATURES.md
$ echo "## T-14: Implement User Login Page" >> PROJECT_FEATURES.md

$ git commit -m "feat: implement user login page (T-01)"
warning: in the working copy of 'PROJECT_FEATURES.md', LF will be replaced by CRLF the next time Git touches it
$ git push -u origin feature/T-01
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 564 bytes | 283.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Create a pull request for 'feature/T-01' on GitHub by visiting:
remote:   https://github.com/Arghya108/Arghya108-project-simulator-Group-4/pull/new/feature/T-01
remote:
remote:   https://github.com/Arghya108/Arghya108-project-simulator-Group-4.git
branch 'feature/T-01' set up to track 'origin/feature/T-01'.

$ git pull origin dev
Already up to date.

$ git checkout stage
Switched to branch 'stage'
Your branch is up to date with 'origin/dev'.

$ git pull origin stage
Already up to date.

$ git checkout dev
Switched to branch 'dev'
Your branch is up to date with 'origin/dev'.

$ git pull origin dev
Already up to date.

$ git checkout -b feature/T-14
Switched to a new branch 'feature/T-14'

$ git branch -d feature/T-14
error: cannot delete branch 'feature/T-14' used by worktree at 'C:/Users/hp/project-simulator/Arghya108-project-simulator-Group-4'

$ git branch -d feature/T-14
error: cannot delete branch 'feature/T-14' used by worktree at 'C:/Users/hp/project-simulator/Arghya108-project-simulator-Group-4'

$ git checkout dev
Switched to branch 'dev'
Your branch is up to date with 'origin/dev'.
```

```
MINGW64/C:/Users/hp/project-simulator/Arghya108-project-simulator-Group-4
$ git checkout dev
Switched to branch 'dev'
Your branch is up to date with 'origin/dev'.

$ git pull origin dev
Already up to date.

$ git checkout stage
Switched to branch 'stage'
Your branch is up to date with 'origin/dev'.

$ git pull origin stage
Already up to date.

$ git checkout dev
Switched to branch 'dev'
Your branch is up to date with 'origin/dev'.

$ git pull origin dev
Already up to date.

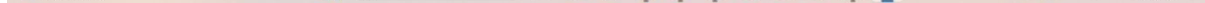
$ git checkout -b feature/T-14
Switched to a new branch 'feature/T-14'

$ git branch -d feature/T-14
error: cannot delete branch 'feature/T-14' used by worktree at 'C:/Users/hp/project-simulator/Arghya108-project-simulator-Group-4'

$ git branch -d feature/T-14
error: cannot delete branch 'feature/T-14' used by worktree at 'C:/Users/hp/project-simulator/Arghya108-project-simulator-Group-4'

$ git checkout dev
Switched to branch 'dev'
Your branch is up to date with 'origin/dev'.
```

```
MINGW64/C:/Users/hp/project-simulator/Arghya108-project-simulator-Group-4
ipbl:APTUP-36882681 WING64 ~/project-simulator
$ git clone https://github.com/Arghya108/Arghya108-project-simulator-Group-4.git
Cloning into 'Arghya108-project-simulator-Group-4'...
fatal: unable to access 'https://github.com/Arghya108/Arghya108-project-simulator-Group-4.git/': Failed to connect to github.com port 443 after 21136 ms: Could not connect to server
ipbl:APTUP-36882681 WING64 ~/project-simulator
$ AC
ipbl:APTUP-36882681 WING64 ~/project-simulator
$ git clone https://github.com/Arghya108/Arghya108-project-simulator-Group-4.git
Cloning into 'Arghya108-project-simulator-Group-4'...
remote: Enumerating objects: 11, done.
remote: Counting objects: 100% (11/11), done.
remote: Compressing objects: 100% (10/10), done.
remote: Total 13 (delta 0), reused 8 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (13/13), 9.13 KiB | 2.56 MiB/s, done.
ipbl:APTUP-36882681 WING64 ~/project-simulator
$ cd project-simulator
bash: cd: project-simulator: No such file or directory
ipbl:APTUP-36882681 WING64 ~/project-simulator
$ cd Arghya108-project-simulator-Group-4
ipbl:APTUP-36882681 WING64 ~/project-simulator/Arghya108-project-simulator-Group-4 (main)
$ git config user.name "ananta-saha"
bash: $ git config user.name "ananta-saha"
ipbl:APTUP-36882681 WING64 ~/project-simulator/Arghya108-project-simulator-Group-4 (main)
$ git config user.name "ananta-saha"
ipbl:APTUP-36882681 WING64 ~/project-simulator/Arghya108-project-simulator-Group-4 (main)
$ git config user.email "anoysaha0606@gmail.com"
ipbl:APTUP-36882681 WING64 ~/project-simulator/Arghya108-project-simulator-Group-4 (main)
$ git checkout main
Already on 'main'
Your branch is up to date with 'origin/main'.
ipbl:APTUP-36882681 WING64 ~/project-simulator/Arghya108-project-simulator-Group-4 (main)
$ git pull origin main
From https://github.com/Arghya108/Arghya108-project-simulator-Group-4
* branch      main       -> FETCH_HEAD
Already up to date.
ipbl:APTUP-36882681 WING64 ~/project-simulator/Arghya108-project-simulator-Group-4 (main)
$ git checkout dev
branch 'dev' set up to track 'origin/dev'.
Switched to a new branch 'dev'
ipbl:APTUP-36882681 WING64 ~/project-simulator/Arghya108-project-simulator-Group-4 (dev)
$ git pull origin dev
From https://github.com/Arghya108/Arghya108-project-simulator-Group-4
* branch      dev        -> FETCH_HEAD
Already up to date.
ipbl:APTUP-36882681 WING64 ~/project-simulator/Arghya108-project-simulator-Group-4 (dev)
```





Ananta saha
ananta-saha

[Edit profile](#)

As 1 follower · 0 following

[in/ananta-saha-08369b2a1](#)

Popular repositories

[GIT_LabTask](#) Public

4 contributions in the last year



Learn how we count contributions

Contribution activity

April 2026

- Created 1 commit in 1 repository
[Arghya108/Arghya108-project-simulator-...](#) 1 commit
- Opened their first pull request on GitHub in Arghya108/Arg... Public Apr 18



First pull request
[feat: implement user login page \(T-01\)](#) Public

Customize your pins

[200k](#)

Arghya108 / Arghya108-project-simulator-Group-4

Code Issues Pull requests Actions Projects Wiki Security and quality Insights Settings

Arghya108-project-simulator-Group-4 Public

Watch Fork Star

main 5 Branches 0 Tags

Go to file Add file Code

ananta-saha feat: implement user login page (T-01) (#4)

CHANGELOG.md chore: initial project setup and documentation yesterday

PROJECT_FEATURES.md feat: implement user login page (T-01) (#4) 30 minutes ago

README.md chore: initial project setup and documentation yesterday

README

"# Project Simulator"

About

No description, website, or topics provided.

Readme

Activity

0 stars

0 watching

0 forks

Report repository

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Contributors 3

- Arghya108 Arghya Das
- Ankon-Saha Ankon Saha
- ananta-saha Ananta saha

Student-3: Clone Repo [Progga Patowary (22-47664-2)]

```
MINOW64/C:/Users/PROGGA PATOWARY/Documents/Arghya108-project-simulator-Group-4
Already up to date.
PROGGA PATOWARY@PROGGA MINGW64 ~/Documents/Arghya108-project-simulator-Group-4 (dev)
$ git checkout stage
branch 'stage' set up to track 'origin/stage'.
Switched to a new branch 'stage'
PROGGA PATOWARY@PROGGA MINGW64 ~/Documents/Arghya108-project-simulator-Group-4 (stage)
$ git pull origin stage
From https://github.com/Arghya108/project-simulator-Group-4
+ branch stage -> FETCH_HEAD
Already up to date.
PROGGA PATOWARY@PROGGA MINGW64 ~/Documents/Arghya108-project-simulator-Group-4 (stage)
$ git checkout dev
Switched to branch 'dev'
Your branch is up to date with 'origin/dev'.
PROGGA PATOWARY@PROGGA MINGW64 ~/Documents/Arghya108-project-simulator-Group-4 (dev)
$ git checkout -b feature group
fatal: 'group' is not a commit and a branch 'feature' cannot be created from it
PROGGA PATOWARY@PROGGA MINGW64 ~/Documents/Arghya108-project-simulator-Group-4 (dev)
$ git add PROJECT-FEATURE.md
fatal: pathspec 'PROJECT-FEATURE.md' did not match any files
PROGGA PATOWARY@PROGGA MINGW64 ~/Documents/Arghya108-project-simulator-Group-4 (dev)
$ git remote add origin git@github.com:proggapatowary-creator/project-simulator-4.git
error: remote origin already exists.
PROGGA PATOWARY@PROGGA MINGW64 ~/Documents/Arghya108-project-simulator-Group-4 (dev)
$ git add .
PROGGA PATOWARY@PROGGA MINGW64 ~/Documents/Arghya108-project-simulator-Group-4 (dev)
$ git commit -m "task submission"
On branch dev
Your branch is up to date with 'origin/dev'.
nothing to commit, working tree clean
PROGGA PATOWARY@PROGGA MINGW64 ~/Documents/Arghya108-project-simulator-Group-4 (dev)
$ git push -u origin main
```

```
MINOW64/C:/Users/PROGGA PATOWARY/Documents/Arghya108-project-simulator-Group-4/project-simulator-group-4
PROGGA PATOWARY@PROGGA MINGW64 ~/Documents/Arghya108-project-simulator-Group-4 (dev)
$ git clone https://github.com/progapatowary-creator/project-simulator-group-4
Cloning into 'project-simulator-group-4'...
remote: Enumerating objects: 14, done.
remote: Counting objects: 100% (14/14), done.
remote: Compressing objects: 100% (10/10), done.
remote: total 14 (delta 1), reused 14 (delta 1), pack-reused 0 (from 0)
Receiving objects: 100% (14/14), 4.55 KiB | 582.00 KiB/s, done.
Resolving deltas: 100% (1/1), done.
PROGGA PATOWARY@PROGGA MINGW64 ~/Documents/Arghya108-project-simulator-Group-4 (dev)
$ cd project-simulator-group-4
PROGGA PATOWARY@PROGGA MINGW64 ~/Documents/Arghya108-project-simulator-Group-4/project-simulator-group-4 (main)
$ git config user.name "Progga Patowary"
PROGGA PATOWARY@PROGGA MINGW64 ~/Documents/Arghya108-project-simulator-Group-4/project-simulator-group-4 (main)
$ git config user.email "proggapatowary@gmail.com"
PROGGA PATOWARY@PROGGA MINGW64 ~/Documents/Arghya108-project-simulator-Group-4/project-simulator-group-4 (main)
$ git checkout main
Already on 'main'
Your branch is up to date with 'origin/main'.
PROGGA PATOWARY@PROGGA MINGW64 ~/Documents/Arghya108-project-simulator-Group-4/project-simulator-group-4 (main)
$ git add q180github.com/progapatowary-creator/project-simulator-4.git
fatal: pathspec 'q180github.com/progapatowary-creator/project-simulator-4.git' did not match any files
PROGGA PATOWARY@PROGGA MINGW64 ~/Documents/Arghya108-project-simulator-Group-4/project-simulator-group-4 (main)
$ git add .
PROGGA PATOWARY@PROGGA MINGW64 ~/Documents/Arghya108-project-simulator-Group-4/project-simulator-group-4 (main)
$ git commit -m "task-submission"
On branch main
Your branch is up to date with 'origin/main'.
nothing to commit, working tree clean
PROGGA PATOWARY@PROGGA MINGW64 ~/Documents/Arghya108-project-simulator-Group-4/project-simulator-group-4 (main)
$ git push
PROGGA PATOWARY@PROGGA MINGW64 ~/Documents/Arghya108-project-simulator-Group-4 (dev)
$ git clone https://github.com/progapatowary-creator/project-simulator-group-4
Cloning into 'project-simulator-group-4'...
remote: Enumerating objects: 14, done.
remote: Counting objects: 100% (14/14), done.
remote: Compressing objects: 100% (10/10), done.
remote: total 14 (delta 1), reused 14 (delta 1), pack-reused 0 (from 0)
Receiving objects: 100% (14/14), 4.55 KiB | 582.00 KiB/s, done.
Resolving deltas: 100% (1/1), done.
PROGGA PATOWARY@PROGGA MINGW64 ~/Documents/Arghya108-project-simulator-Group-4 (dev)
$ cd project-simulator-group-4
PROGGA PATOWARY@PROGGA MINGW64 ~/Documents/Arghya108-project-simulator-Group-4/project-simulator-group-4 (main)
$ git config user.name "Progga Patowary"
```



```
MINGW64/c/Users/PROGGA PATOWARY/Documents/Arghya108-project-simulator-Group-4/project-simulator-group-4
fatal: pathspec 'git@github.com:proggapatowary-creator/project-simulator-4.git' did not match any files
PROGGA PATOWARY@PROGGA MINGW64 ~/Documents/Arghya108-project-simulator-Group-4/project-simulator-group-4 (main)
$ git add .
PROGGA PATOWARY@PROGGA MINGW64 ~/Documents/Arghya108-project-simulator-Group-4/project-simulator-group-4 (main)
$ git commit -m "task-submission"
On branch main
Your branch is up to date with 'origin/main'.
nothing to commit, working tree clean
PROGGA PATOWARY@PROGGA MINGW64 ~/Documents/Arghya108-project-simulator-Group-4/project-simulator-group-4 (main)
$ git push
PROGGA PATOWARY@PROGGA MINGW64 ~/Documents/Arghya108-project-simulator-Group-4/project-simulator-group-4 (dev)
$ git clone https://github.com/proggapatowary-creator/project-simulator-group-4
Cloning into 'project-simulator-group-4'...
remote: Enumerating objects: 14, done.
remote: Counting objects: 100% (14/14), done.
remote: Compressing objects: 100% (10/10), done.
remote: Total 14 (delta 1), reused 14 (delta 1), pack-reused 0 (from 0)
Receiving objects: 100% (14/14), 4.55 KiB | 582.00 KiB/s, done.
Resolving deltas: 100% (1/1), done.
PROGGA PATOWARY@PROGGA MINGW64 ~/Documents/Arghya108-project-simulator-Group-4 (dev)
$ cd project-simulator-group-4
PROGGA PATOWARY@PROGGA MINGW64 ~/Documents/Arghya108-project-simulator-Group-4/project-simulator-group-4 (main)
$ git config user.name "Progga-Patowary"
PROGGA PATOWARY@PROGGA MINGW64 ~/Documents/Arghya108-project-simulator-Group-4/project-simulator-group-4 (main)
$ git config user.email "proggapatowary@gmail.com"
PROGGA PATOWARY@PROGGA MINGW64 ~/Documents/Arghya108-project-simulator-Group-4/project-simulator-group-4 (main)
$ git checkout main
Already on 'main'
Your branch is up to date with 'origin/main'.
PROGGA PATOWARY@PROGGA MINGW64 ~/Documents/Arghya108-project-simulator-Group-4/project-simulator-group-4 (main)
$ git add git@github.com:proggapatowary-creator/project-simulator-4.git
fatal: pathspec 'git@github.com:proggapatowary-creator/project-simulator-4.git' did not match any files
PROGGA PATOWARY@PROGGA MINGW64 ~/Documents/Arghya108-project-simulator-Group-4/project-simulator-group-4 (main)
$ git add .
PROGGA PATOWARY@PROGGA MINGW64 ~/Documents/Arghya108-project-simulator-Group-4/project-simulator-group-4 (main)
$ git commit -m "task-submission"
On branch main
Your branch is up to date with 'origin/main'.
nothing to commit, working tree clean
PROGGA PATOWARY@PROGGA MINGW64 ~/Documents/Arghya108-project-simulator-Group-4/project-simulator-group-4 (main)
$ git push
$ git push git@github.com:proggapatowary-creator/project-simulator-4.git
```

Student-4 : Review [Ankon Saha (23-54172-3)]

feat: implement user login page (T-01) #4

ananta-saha wants to merge 1 commit into main from feature/T-01

Conversation 1 | Commits 1 | Checks 0 | Files changed 1

ananta-saha commented 30 minutes ago

Created login form component with fields - Added input validation - Implemented error handling for failed login attempts

feat: implement user login page (T-01)

ananta-saha requested a review from Ankon-Saha 29 minutes ago

Ankon-Saha approved these changes 23 minutes ago

Ankon-Saha left a comment

wonderful project

github.com/Arghya108/.../changes

Reviewers: Ankon-Saha

Assignees: No one—assign yourself

Labels: None yet

Projects: None yet

Milestone: No milestone

Development: Successfully merging this pull request may close these issues. None yet

6. SOFTWARE TESTING

Project Name: Integrated Sports Information and Management System for AIUB			Test Designed by: Arghya Das	
Test Case ID: TC_1			Test Designed date: 02/5/2026	
Test Priority (Low, Medium, High): High			Test Executed by: Arghya Das	
Module Name: Forgot Password			Test Execution date: 05/06/2026	
Test Title: Verification of Password Recovery via Email OTP/Link				
Description: Verifies that a user can reset their password through a verified email process and that the old password becomes invalid immediately.				
Precondition: A user account exists.				
Dependencies: SMTP Server (Email Service) must be configured and active in the system.				
Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
1. Click "Forgot Password" on the Login page. 2. Enter registered Student/Admin ID or Email. 3. Check email inbox for recovery link/OTP. 4. Click link and enter a new password. 5. Attempt to login with the old password. 6. Attempt to login with the new password.	ID: 23-54149-3 New pass: fdyguh	System validates password strength and saves the new hash and meanwhile delete the old pass	As expected	Pass

Project Name: Integrated Sports Information and Management System for AIUB			Test Designed by: Arghya Das	
Test Case ID: TC_2			Test Designed date: 02/5/2026	
Test Priority (Low, Medium, High): High			Test Executed by: Arghya Das	
Module Name: Payment Info			Test Execution date: 05/07/2026	
Test Title: Verification of Online Payment Processing and Confirmation.				
Description: Verifies that a user can reset their password through a verified email process and that the old password becomes invalid immediately.				
Precondition: User must be logged in and have an active registration or membership request pending.				
Dependencies: Payment gateway must be configured and active in the system.				
Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
1. Navigate to the Payment section from the dashboard. 2. Enter recipient address, amount, and re-type amount. 3. Click "Sign with Private Key" button. 4. Click Submit button to process payment. 5. Admin confirms the payment from the admin panel. 6. Check user payment history for the transaction record.	ID: 23-54149-3 Amount: 500 BDT	payment page loads with all fields visible. All fields accept valid input without error. System signs the transaction and enables Submit button. Transaction is added to the system and status shows "Pending". Payment status updates to "Confirmed" in real time. Transaction appears in history with correct amount and date.	Page loads but payment fields do not respond to input. Fields visible but form submission not working. Button does not respond on click. Submit button non-functional. No response from system. Cannot test — payment never submitted. No record generated.	Fail

Project Name: Integrated Sports Information and Management System for AIUB			Test Designed by: Ananta Saha	
Test Case ID: TC_1			Test Designed date: 01/5/2026	
Test Priority (Low, Medium, High): High			Test Executed by: Ananta Saha	
Module Name: Social Login (Microsoft Teams)			Test Execution date: 17/06/2026	
Test Title: Verification of Login via Microsoft Teams Account				
Description: Verifies that a user can successfully log in using their Microsoft Teams account and access the system without manually entering system credentials. Ensures proper authentication and account linking.				
Precondition: User must have a valid Microsoft account linked with Microsoft Teams. Microsoft authentication (OAuth 2.0 / Azure AD) must be configured in the system.				
Dependencies: Internet connection. Microsoft Azure Active Directory (AAD) service must be active.				
Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
1. Click on “Login with Microsoft Teams” button on the signup page. 2. System redirects to Microsoft login page. 3. Enter valid Microsoft Teams credentials. 4. Grant permission (if prompted). 5. System retrieves user information (name, email, ID). 6. User is redirected back to the system dashboard.	Email: user@outlook.com	User is successfully authenticated via Microsoft account. If first-time login, system creates a new user profile using retrieved data. If existing user, system logs in directly. User is redirected to dashboard without errors.	As expected	Pass

Project Name: Integrated Sports Information and Management System for AIUB			Test Designed by: Ananta Saha	
Test Case ID: TC_2			Test Designed date: 01/5/2026	
Test Priority (Low, Medium, High): High			Test Executed by: Ananta Saha	
Module Name: Schedule Management			Test Execution date: 17/06/2026	
Test Title: Verification of Cancel Activity from Weekly Schedule.				
Description: Verifies that a user can cancel/remove a scheduled activity (training/match/practice) and that the activity is no longer visible in the schedule.				
Precondition: User account exists and is logged in. At least one activity is already scheduled in the weekly schedule.				
Dependencies: Active database connection. Schedule module must be functional.				
Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
1. Navigate to Schedule page. 2. Select a scheduled activity (e.g., Training/Match). 3. Click on Cancel/Delete Activity option. 4. Confirm cancellation (if confirmation dialog appears). 5. Refresh or revisit the schedule page.	Activity: MATCH (May 4, 5:00 PM – 6:50 PM)	The selected activity is removed from the schedule. The updated schedule is displayed without the cancelled activity. A confirmation message is shown (e.g., “Activity cancelled successfully”).	As expected	Pass

Project Name: Integrated Sports Information and Management System for AIUB			Test Designed by: Progga Patowary	
Test Case ID: TC_1			Test Designed date: 02/5/2026	
Test Priority (Low, Medium, High): Medium			Test Executed by: Progga Patowary	
Module Name: About Us Page			Test Execution date: 25/05/2026	
Test Title: Verify About Us Page Content Display.				
Description: This test checks whether the About Us page loads correctly and displays all necessary information about the system.				
Precondition: User has access to the system.				
Dependencies: Internet connection is active.				
Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
1. Open the website. 2. Click on “About Us” from the navigation bar.	Observe the page content.	About Us page loads successfully. All text, images, and descriptions are visible. No broken layout or missing content.	Page loaded successfully with all content visible.	Pass

Project Name: Integrated Sports Information and Management System for AIUB			Test Designed by: Progga Patowary	
Test Case ID: TC_2			Test Designed date: 02/5/2026	
Test Priority (Low, Medium, High): High			Test Executed by: Progga Patowary	
Module Name: About Us Page			Test Execution date: 25/05/2026	
Test Title: Verify Contact Form Submission.				
Description: This test verifies that users can send messages through the Contact form.				
Precondition: User is on Contact page.				
Dependencies: Internet connection is active.				
Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)

1. Open Contact page. 2. Fill in Name, Email, Message. 3. Click "Send Message".	Observe the page content.	Message is submitted successfully. Confirmation message is shown.	Message submitted and confirmation displayed.	Passed
---	---------------------------	---	---	--------

Project Name: Integrated Sports Information and Management System for AIUB			Test Designed by: Ankon saha	
Test Case ID: TC_1			Test Designed date: 02/5/2026	
Test Priority (Low, Medium, High): High			Test Executed by: Ankon Saha	
Module Name: Integrated Sports Dashboard			Test Execution date: 05/06/2026	
Test Title: Verification of Comprehensive Sports Information (Live, Upcoming, & Results)				
Description: Verifies that the user can view real-time live scores, access detailed tournament schedules (including semifinals and finals) via "View Details," and check historical match results and team information.				
Precondition: The system requires an authenticated user and a database populated with live matches, upcoming tournament schedules, and historical results to function correctly.				
Dependencies: Real-time API/Socket updates and tournament bracket retrieval require active internet connectivity and an operational database server.				
Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
1. Click on a specific live match from the "Live" tab. A detailed live match page opens showing score breakdown, player stats, and match events. 2. Refresh the dashboard during a live match. Live scores update correctly without data loss or duplication. 3.Click "View Details" for detailed page with	Same live match, Events, Team List	Real-time score updates (e.g., 156/4 in 18 overs) are displayed accurately. System redirects to a detailed view showing match dates and participating teams. A chronological list of completed matches with final scores and winners is displayed. The system displays all registered teams and their member details.	As expected	Pass

schedule, teams (12 teams), and tournament stages. 4. Check "Registration Open" status for upcoming event. Same event Badge "Registration Open" is clearly visible and correctly labeled. 5. When the user selects the "Teams" tab, the system displays a comprehensive list of all registered teams and their respective squad members across all sports categories.		The dashboard correctly displays the start dates and registration status for future sports.		
---	--	---	--	--

Project Name: Integrated Sports Information and Management System for AIUB			Test Designed by: Ankon saha	
Test Case ID: TC_2			Test Designed date: 02/5/2026	
Test Priority (Low, Medium, High): High			Test Executed by: Ankon Saha	
Module Name: Team Management			Test Execution date: 05/06/2026	
Test Title: Verification of Team List View and Join Functionality				
Description: Verifies that users can view all registered sports teams with member counts and successfully initiate a join request.				
Precondition: The system requires an authenticated user and a database populated with live matches, upcoming tournament schedules, and historical results to function correctly.				
Dependencies: Real-time API/Socket updates and tournament bracket retrieval require active internet connectivity and an operational database server.				
Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
1. Navigate to the main "Teams" or "Players" section. 2. Inspect individual team cards for accuracy.	Cricket, Volleyball, Team	The system displays a comprehensive list of all registered teams and their respective squad members across all sports categories.	As expected	Pass

3. Click the "Join" button on a specific team card.		System triggers a join request or opens a confirmation dialog for the user to join the basketball team.		
4. Scroll through the list to ensure all categories are present.		Member counts for each team are updated and displayed clearly under the team name.		
5. Check if member counts reflect the current database state.		The system successfully redirects the user to the respective event or match management modules.		
6. Click on "Events" or "Matches" from the blue sub-header.				

7. CONCLUSION

The Integrated Sports Information and Management System for AIUB is an important solution for the current problems in sports management. This system helps to store and manage all sports information in one place. Students can easily see event schedules, register for tournaments, and get important updates. They do not need to visit the sports office again and again. This system saves time and makes the process simple and fast. For administrators, the system helps to manage events, payments, and records in an organized way. It also reduces confusion and errors in manual work. The use of the Agile Scrum model helps the team to develop the system step by step and improve it with feedback. The system also supports better communication between students and the sports department. However, due to limited time, not all features may be fully developed. In the future, more features can be added and improved. Overall, this system is useful, easy to use, and helpful for both students and administrators. It can improve the sports management system of AIUB in a better and more modern way.